Attributes

We've not dived too deeply into HTML attributes yet, but you have already seen them. `<meta charset="utf-8" />` is a `meta` element with a `charset` attribute. All HTML attributes follow the form `name="value"` and are placed inside the tag, after the tag name, separated by at least one whitespace/tab/new line character. An HTML element can have any number of attributes, many of which are predefined and have special meaning. There are many predefined attributes, some of which are "global" (can apply to any element), such as the important `class` (which is used to drive CSS styling and JavaScript) and `id` (a unique ID string given to just one element in the entire DOM, again to drive styling and behavior). Some predefined attributes are specific to a tag; for example, the `http-equiv` attribute only has meaning inside a `meta` element. "Custom" attributes – that have no special predetermined effect on rendering or behavior by default – are often used to drive front-end libraries and frameworks (the Angular.io TypeScript framework, for example, uses HTML attributes prefixed with `ng-`).

Attributes are exposed by the `dom` API in much the same way as nodes: via public property. We can access the attributes of the `meta` element in our example HTML like so:

```
$metaNodes = $dom->getElementsByTagName('meta');

foreach ($metaNodes as $metaNode) {
    if ($metaNode->nodeType === XML_ELEMENT_NODE) {
        foreach ($metaNode->attributes as $attr) {
            var_dump(get_class($attr));
            var_dump($attr->name);
            var_dump($attr->value);
        }
    }
}

Output:
string(8) "Dom\Attr"
string(7) "charset"
string(5) "utf-8"
```

Here we make use of the method `Dom\HTMLDocument::getElementsByTagName()`, which will traverse the entire DOM tree and return an array of all matching element nodes. This method is also inherited by any class in the `dom` API that extends `Dom\Element`.